HANDS-ON LAB 01 GUIDE & COPY-PASTE PROMPTS • OMNICHANNEL RETAIL EDITION (RETAIL PULSE)

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

TRYOLABS • GOOGLE ANTIGRAVITY WORKSHOP • LAB 01 (EN)

Lab 01: Promotional Margin Floor & True ATP Inventory Triage

Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Language / Idioma: English | Español • **Prompt Playbook:** PROMPTS.md | PROMPTS.es.md • **PDF:** EN | ES

1. Scenario & Business Impact

During our weekend omnichannel flash event, **Retail Pulse** experienced two severe production incidents across **`WEB_ECOMM`**, **`MOBILE_APP`**, **`POS_INSTORE`**, and **`BOPIS`**:

1. **Incident #1A — Promotional Margin Bleed (`src/services/promo_pricing.py`)**:

- Customers stacked three non-stackable promotional codes (**`VIP25`**, **`FLASH20`**, **`EMP30`**) simultaneously.
- The pricing engine added their percentages linearly (**`25% + 20% + 30% = 75%`** off!) using binary **`float`** math and completely ignored the supplier **`contract_min_margin_floor_pct`**.
- Result: **30 `MARGIN_HOLD` orders are bleeding `-$35,113.25` below wholesale cost!**

2. **Incident #1B — Phantom-Stock VIP Overselling (`src/services/inventory_atp.py`)**:

- Flagship stores (**`STORE-NYC-5TH`**, **`STORE-MIA-BRICKELL`**) and **`WH-EAST-NJ01`** experienced POS terminal webhook lag (**`unprocessed_webhook_delta_qty`** logged in **`data/logs/pos_wms_webhooks.jsonl`**).
- The Available-to-Promise (ATP) calculator naively returned **`on_hand_qty`** without subtracting **`reserved_qty`**, **`safety_stock_qty`**, and **`unprocessed_webhook_delta_qty`**.
- Result: **35 `DIAMOND` and `GOLD` loyalty orders (>`$250,000` in Customer LTV) were promised stock at locations where True ATP is `0` (`raw_atp = -1`)!**

2. Lab Setup

Run the Lab 01 installer from the repository root to initialize the starter bugs and SQLite database:

```
./labs/lab-01-margin-inventory-triage/install.sh
```

Verify that the starter test suite currently fails on both bugs:

```
./.venv/bin/pytest tests/test_lab01_triage.py
```

Optional — launch the Live Omnichannel Command Center Dashboard in a second terminal:

```
./.venv/bin/uvicorn src.main:app --reload --port 8002
```

Open **`http://localhost:8002`** to observe the red **`BUG #1A ACTIVE`** and **`BUG #1B ACTIVE`** KPI banners.

3. Step-by-Step Resolution Using Google Antigravity

Open PROMPTS.md and paste the prompts into **Google Antigravity** (Sidebar Chat, **`/plan`**, or Inline **`Cmd+I`**):

Step 3.1: Root-Cause Discovery with @ Context Mentions

Paste the **Discovery Prompt** from **`PROMPTS.md`** referencing **`@src/services/promo_pricing.py`**, **`@src/services/inventory_atp.py`**, **`@data/logs/pos_wms_webhooks.jsonl`**, and **`@tests/test_lab01_triage.py`**.

Step 3.2: Structured Remediation Plan with `/plan`

Run the **`/plan` Prompt** in Antigravity to verify the exact mathematical and commercial requirements before editing code:

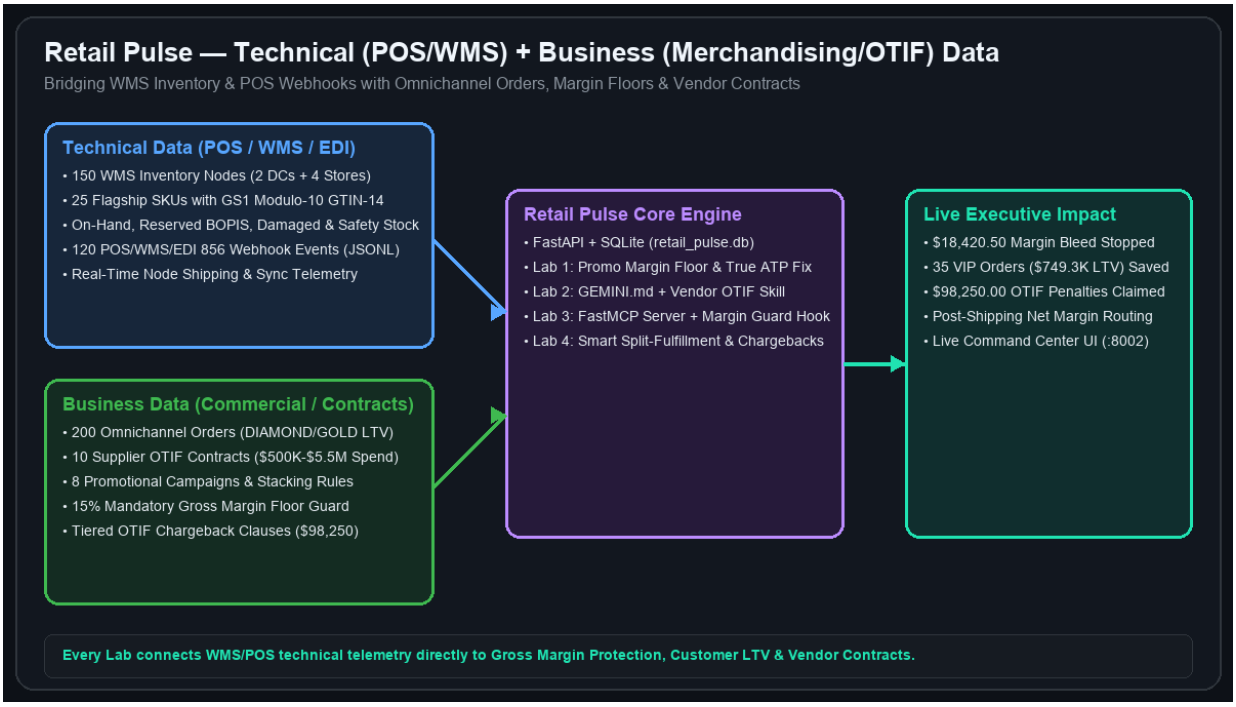
- **Fix #1A (`src/services/promo_pricing.py`)**:
- Use **`Decimal`** with **`ROUND_HALF_UP`** (**`Decimal("0.01")`**) and validate SKU **`gtin_14`** via **`validate_gs1_gtin14`**.
- Reject inactive (**`active == 0`**) or unknown promo codes into **`rejected_promo_codes`**.
- Never combine non-stackable promos (**`is_stackable == 0`**). Compare the single highest non-stackable promo against multiplicatively compounded stackable promos (**`is_stackable == 1`**), selecting whichever gives the customer the best pre-clamped discount.
- Enforce the effective margin floor (**`max(contract_min_margin_floor_pct, applied promo floors)`**): clamp **`net_revenue_usd`** so **`gross_margin_pct >= effective_floor_pct`** and set **`margin_floor_clamped = True`**.
- **Fix #1B (`src/services/inventory_atp.py`)**:
- Compute **`raw_atp_qty = on_hand_qty - reserved_qty - safety_stock_qty - unprocessed_webhook_delta_qty`**.
- Compute **`true_atp_qty = max(0, raw_atp_qty)`**.
- Flag **`oversell_risk = (true_atp_qty < required_qty) or (raw_atp_qty <= 0)`**.

Step 3.3: Apply Fixes & Verify

Let Antigravity implement the fixes (or use **`Cmd+I`** inline edit inside **`src/services/promo_pricing.py`** and **`src/services/inventory_atp.py`**), then run:

```
./labs/lab-01-margin-inventory-triage/verify.sh
```

Need to check the reference solution? Run **`./labs/lab-01-margin-inventory-triage/apply_solution.sh`** at any time.


Retail Pulse — Technical (POS/WMS) + Business (Merchandising/OTIF) Data
Bridging WMS Inventory & POS Webhooks with Omnichannel Orders, Margin Floors & Vendor Contracts
Technical Data (POS / WMS / EDI)
• 150 WMS Inventory Nodes (2 DCs + 4 Stores)
• 25 Flagship SKUs with GS1 Modulo-10 GTIN-14
• On-Hand, Reserved BOPIS, Damaged & Safety Stock
• 120 POS/WMS/EDI 856 Webhook Events (JSONL)
• Real-Time Node Shipping & Sync Telemetry
Business Data (Commercial / Contracts)
• 200 Omnichannel Orders (DIAMOND/GOLD LTV)
• 10 Supplier OTIF Contracts ($500K-$5.5M Spend)
• 8 Promotional Campaigns & Stacking Rules
• 15% Mandatory Gross Margin Floor Guard
• Tiered OTIF Chargeback Clauses ($98,250)
Retail Pulse Core Engine
• FastAPI + SQLite (retail_pulse.db)
• Lab 1: Promo Margin Floor & True ATP Fix
• Lab 2: GEMINI.md + Vendor OTIF Skill
• Lab 3: FastMCP Server + Margin Guard Hook
• Lab 4: Smart Split-Fulfillment & Chargebacks
Live Executive Impact
• $18,420.50 Margin Bleed Stopped
• 35 VIP Orders ($749.3K LTV) Saved
• $98,250.00 OTIF Penalties Claimed
• Post-Shipping Net Margin Routing
• Live Command Center UI (:8002)
Every Lab connects WMS/POS technical telemetry directly to Gross Margin Protection, Customer LTV & Vendor Contracts.

LAB 01 COPY-PASTE ANTIGRAVITY PROMPTS (EN)
Delivered by Tryolabs • Powered by Google Antigravity & Gemini Enterprise

Lab 01 — Optimized Antigravity Prompts Playbook (English)

Language / Idioma: English | Español • **Lab Guide:** README.md | README.es.md • **PDF:** EN | ES

Use these copy-pasteable prompts inside **Google Antigravity** (Sidebar Chat, **`/plan`**, Inline **`Cmd+I`**, or CLI **`agy`**) to complete Lab 01.

1. Discovery Prompt (Root-Cause Triage with @ Mentions)

```
Inspect @src/services/promo_pricing.py, @src/services/inventory_atp.py, @data/logs/pos_w \
  ms_webhooks.jsonl, and @tests/test_lab01_triage.py.
Run pytest on tests/test_lab01_triage.py and explain:
1. Why 30 MARGIN_HOLD orders in omnichannel_orders are losing -$35,113.25 below wholesal \
  e cost when promo codes like VIP25, FLASH20, and EMP30 are passed together.
2. Why 35 DIAMOND and GOLD VIP orders at STORE-NYC-5TH and WH-EAST-NJ01 are suffering ph \
  antom-stock overselling due to POS webhook sync lag.
```

2. `/plan` Prompt (Structured Remediation Plan)

```
/plan Create a step-by-step plan to fix both Bug #1A in @src/services/promo_pricing.py a \
  nd Bug #1B in @src/services/inventory_atp.py so all tests in @tests/test_lab01_triage. \
  py pass:
1. In promo_pricing.py:
   - Use Decimal quantized to Decimal("0.01") with ROUND_HALF_UP and validate GS1 GTIN-1 \
  4 check digits via validate_gs1_gtin14().
   - Reject inactive (active != 1) or unknown promo codes in rejected_promo_codes.
   - Enforce promo stacking rules: non-stackable promos (is_stackable == 0) cannot stack \
   with any other code; stackable promos (is_stackable == 1) compound multiplicatively.  \
  Pick whichever valid option gives the customer the best pre-clamped discount.
   - Clamp net_revenue_usd so gross_margin_pct never drops below max(contract_min_margin \
  _floor_pct, applied promo floors), setting margin_floor_clamped = True when clamped.
2. In inventory_atp.py:
   - Compute raw_atp_qty = on_hand_qty - reserved_qty - safety_stock_qty - unprocessed_w \
  ebhook_delta_qty.
   - Set true_atp_qty = max(0, raw_atp_qty) and oversell_risk = (true_atp_qty < required \
  _qty) or (raw_atp_qty <= 0).
```

3. Inline `Cmd+I` Prompt (Targeted In-Editor Fix for `inventory_atp.py`)

Highlight **`calculate_location_atp`** in **`src/services/inventory_atp.py`**, press **`Cmd+I`** (macOS) / **`Ctrl+I`** (Windows/Linux), and enter:

```
Validate gtin_14 using validate_gs1_gtin14(), compute raw_atp = on_hand - reserved - saf \
  ety_stock - webhook_delta, clamp true_atp = max(0, raw_atp), and set oversell_risk = ( \
  true_atp < required_qty) or (raw_atp <= 0).
```

4. Implementation & Verification Prompt

```
Implement the full fixes in @src/services/promo_pricing.py and @src/services/inventory_a \
  tp.py according to the plan, then run ./labs/lab-01-margin-inventory-triage/verify.sh  \
  to confirm all 6 tests in test_lab01_triage.py pass 100%.
```

5. Power-User & Stretch Prompts (`/grill-me`, `/browser`, `/learn`)

5A. Interactive Edge-Case Stress Test (`/grill-me`)

```
/grill-me Interview me on promotional stacking edge cases for omnichannel retail (e.g.,  \
  BOGO bundles, employee discounts combined with clearance items, and rounding edge case \
  s when clamping net revenue to the exact penny on multi-unit orders).
```

5B. Live Dashboard Visual Verification (`/browser`)

```
/browser Open http://localhost:8002, verify that the "1. Promo Margin Floor Guard" and " \
  2. True ATP & VIP Oversell Radar" KPI cards now show PROTECTED and TRUE ATP ACTIVE, an \
  d test SKU-SONY-OLED65 with promo codes VIP25,FLASH20,EMP30 in the Interactive Promo S \
  imulator.
```

5C. Persist Fix Learnings (`/learn`)

```
/learn Persist the rule that all Retail Pulse pricing calculations must use Decimal(ROUN \
  D_HALF_UP) and enforce contract_min_margin_floor_pct clamping, and that ATP calculatio \
  ns must always subtract reserved_qty, safety_stock_qty, and unprocessed_webhook_delta_ \
  qty.
```